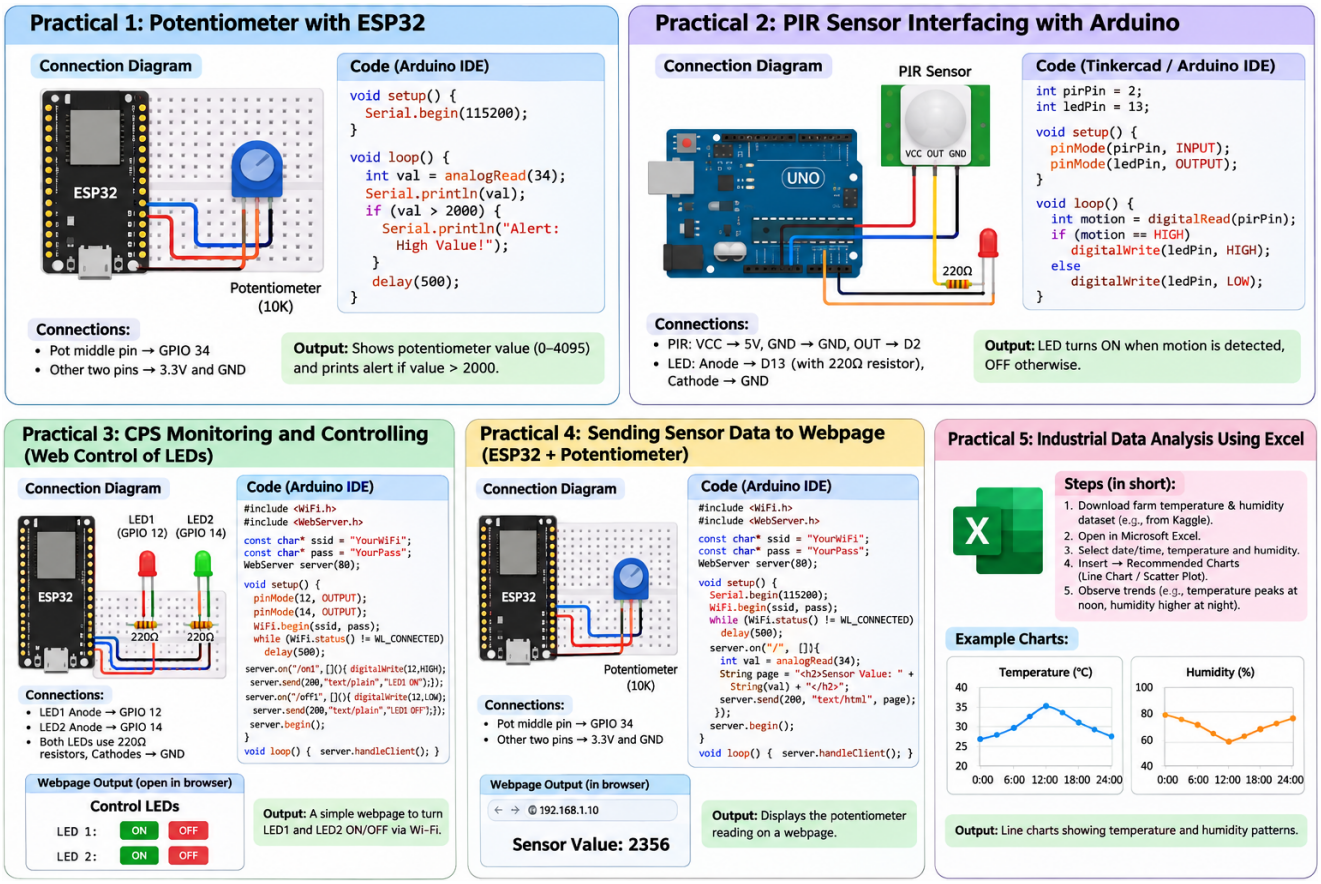
IIOT External Viva & Practical Guide

Computer Laboratory II – Industrial Internet of Things | Course Code: 417526

Simple Arduino / ESP32 programs, connection references, execution steps, explanations and expected outputs for all five practicals.



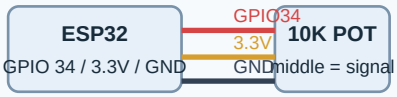
How to use this guide: First learn the aim and connections, then understand the short code, and finally practise explaining the output in your own words.

Important: The programs below are simplified, viva-friendly versions based on the practicals in your lab manual. Practical 3 and 4 use a local ESP32 web server; Practical 5 is an Excel activity, not an Arduino program.

Practical 1 – Potentiometer with ESP32

Aim: Read potentiometer values using ESP32 and print a warning when the reading exceeds 2000.

Practical 1 – ESP32 + Potentiometer



Connect the potentiometer's two outer terminals to 3.3V and GND; middle terminal to GPIO 34.

Components and connections

Component / pin	Connection
10K potentiometer middle pin	ESP32 GPIO 34
Potentiometer outer pin 1	ESP32 3.3V
Potentiometer outer pin 2	ESP32 GND
Other components	ESP32 board, breadboard, jumper wires, USB data cable

Arduino IDE code

```
const int potPin = 34;
int potValue = 0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  potValue = analogRead(potPin);
  Serial.println(potValue);

  if (potValue > 2000) {
    Serial.println("WARNING!");
  }

  delay(500);
}
```

How to explain the code

1. **potPin = 34** stores the ESP32 pin connected to the potentiometer.
2. **Serial.begin(115200)** starts serial communication.
3. **analogRead(potPin)** reads the analog input.
4. **Serial.println()** displays the reading.
5. The **if** condition prints a warning when the reading is above 2000.
6. **delay(500)** waits half a second.

Execution steps

Connect the potentiometer; select the correct ESP32 board and COM port in Arduino IDE; upload the sketch; open Serial Monitor at 115200 baud; rotate the potentiometer and observe the readings and warning.

Expected output

```
1050\n1500\n2100\nWARNING!\n2500\nWARNING!
```

Viva points: Potentiometer = variable resistor; ADC = Analog-to-Digital Converter; ESP32 12-bit reading range used here = 0–4095; GPIO 34 is input-only and supports analog input.

Practical 2 – PIR Sensor Interfacing with Arduino

Aim: Detect motion using a PIR sensor and switch an LED ON or OFF.



Components and connections

Component / pin	Arduino connection
PIR VCC	5V
PIR GND	GND
PIR OUT	Digital pin 2
LED anode (+)	Digital pin 13 through 220Ω resistor
LED cathode (–)	GND
Other components	Arduino board, breadboard and jumper wires

Arduino IDE / Tinkercad code

```
int pirPin = 2;
int ledPin = 13;

void setup() {
  pinMode(pirPin, INPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int motion = digitalRead(pirPin);

  if (motion == HIGH) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Motion Detected!");
  }
  else {
    digitalWrite(ledPin, LOW);
    Serial.println("No Motion");
  }

  delay(1000);
}
```

How to explain the code

1. **pirPin = 2** defines the PIR output pin; **ledPin = 13** defines the LED pin.
2. **pinMode()** configures the sensor as INPUT and LED as OUTPUT.
3. **digitalRead()** reads HIGH or LOW from the PIR sensor.

4. If motion is HIGH, **digitalWrite()** turns the LED ON.
5. Otherwise, the LED is turned OFF.
6. Serial messages show the detected state.

Execution steps

Create a circuit in Tinkercad or wire the Arduino; connect PIR VCC to 5V, GND to GND and OUT to D2; connect the LED with a series resistor; paste the code in Text mode; start the simulation or upload the sketch; test motion.

Expected output

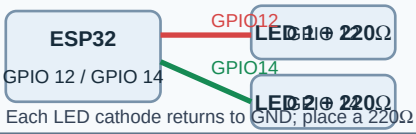
```
Motion Detected!\nNo Motion\nMotion Detected!
```

Viva points: PIR = Passive Infrared; it detects changes in infrared radiation caused by movement. HIGH means motion detected and LOW means no motion detected. PIR sensors are used in security alarms and automatic lighting.

Practical 3 – CPS Monitoring and Control using ESP32

Aim: Create a simple webpage to control two LEDs connected to an ESP32 over Wi-Fi.

Practical 3 – ESP32 + Two LEDs



Each LED cathode returns to GND; place a 220Ω resistor in series with each LED.

Components and connections

Component / pin	ESP32 connection
LED 1 anode (+)	GPIO 12 through 220Ω resistor
LED 1 cathode (–)	GND
LED 2 anode (+)	GPIO 14 through 220Ω resistor
LED 2 cathode (–)	GND
Other components	ESP32, breadboard, jumper wires, USB cable and Wi-Fi

Arduino IDE code

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "YourWiFi";
const char* password = "YourPassword";

WebServer server(80);

void setup() {
  Serial.begin(115200);
  pinMode(12, OUTPUT);
  pinMode(14, OUTPUT);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }

  Serial.println(WiFi.localIP());

  server.on("/", []() {
    String page = "<h2>LED Control</h2>";
    page += "<a href='/12/on'>LED 1 ON</a><br>";
    page += "<a href='/12/off'>LED 1 OFF</a><br>";
    page += "<a href='/14/on'>LED 2 ON</a><br>";
    page += "<a href='/14/off'>LED 2 OFF</a>";
    server.send(200, "text/html", page);
  });

  server.on("/12/on", []() {
    digitalWrite(12, HIGH);
    server.send(200, "text/plain", "LED 1 ON");
  });
  server.on("/12/off", []() {
    digitalWrite(12, LOW);
    server.send(200, "text/plain", "LED 1 OFF");
  });
  server.on("/14/on", []() {
```

```
    digitalWrite(14, HIGH);
    server.send(200, "text/plain", "LED 2 ON");
  });
  server.on("/14/off", []() {
    digitalWrite(14, LOW);
    server.send(200, "text/plain", "LED 2 OFF");
  });

  server.begin();
}

void loop() {
  server.handleClient();
}
```

How to explain the code

1. **WiFi.h** connects ESP32 to a Wi-Fi network.
2. **WebServer.h** provides web-server functions.
3. **WebServer server(80)** creates a server on HTTP port 80.
4. **WiFi.begin()** starts the Wi-Fi connection.
5. **WiFi.localIP()** prints the ESP32 IP address.
6. **server.on()** defines URL routes for ON/OFF actions.
7. **digitalWrite()** controls each LED.
8. **server.handleClient()** processes browser requests.

Execution steps

Replace YourWiFi and YourPassword with the network credentials; wire the LEDs and resistors; select the ESP32 board and port; upload the code; open Serial Monitor at 115200 baud; enter the printed IP address in a browser connected to the same network; click the LED links.

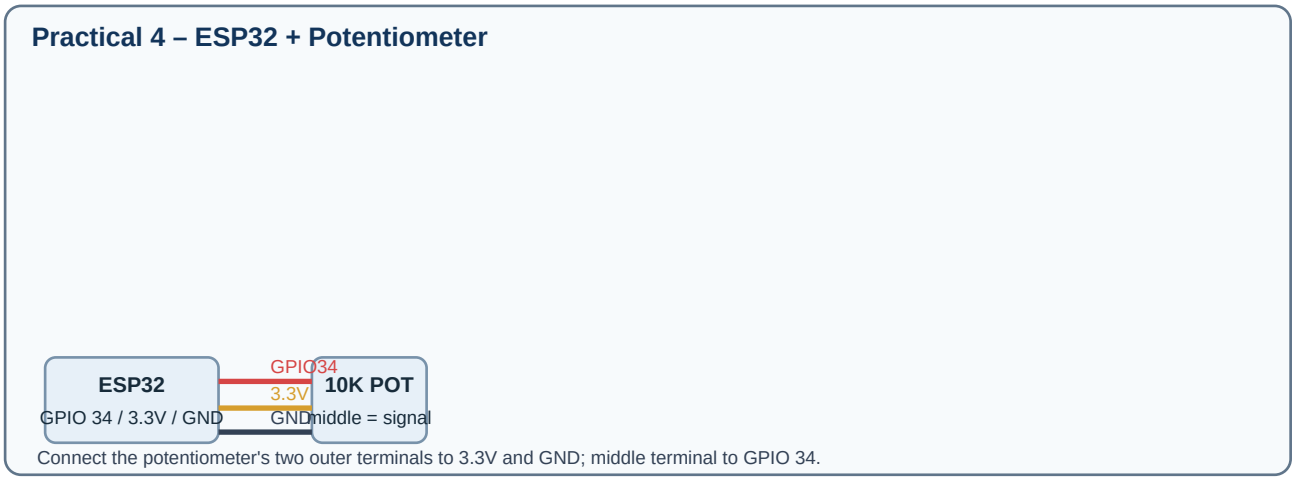
Expected output

A webpage titled LED Control appears with four links: LED 1 ON, LED 1 OFF, LED 2 ON and LED 2 OFF. Clicking a link changes the corresponding LED state.

Viva points: CPS = Cyber-Physical System; HTTP = HyperText Transfer Protocol; HTML = HyperText Markup Language; IP address identifies a device on a network; a 220Ω resistor limits LED current.

Practical 4 – Sending Sensor Data to a Webpage

Aim: Read a potentiometer value using ESP32 and display it on a webpage hosted by ESP32.



Components and connections

Potentiometer pin	ESP32 connection
Middle pin	GPIO 34
Outer pin 1	3.3V
Outer pin 2	GND
Other components	ESP32, breadboard, jumper wires, USB cable and Wi-Fi

Arduino IDE code

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "YourWiFi";
const char* password = "YourPassword";

WebServer server(80);
int potPin = 34;
int potValue = 0;

void handleRoot() {
  potValue = analogRead(potPin);

  String page = "<h2>Sensor Value: ";
  page += String(potValue);
  page += "</h2>";
  page += "<meta http-equiv='refresh' content='2'>";

  server.send(200, "text/html", page);
}

void setup() {
  Serial.begin(115200);
  pinMode(potPin, INPUT);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }

  Serial.println(WiFi.localIP());

  server.on("/", HTTP_GET, handleRoot);
  server.begin();
}

void loop() {
  server.handleClient();
```

```
}
```

How to explain the code

1. **WiFi.h** enables Wi-Fi connectivity; **WebServer.h** creates the web server.
2. **WebServer server(80)** creates a server on port 80.
3. **analogRead()** reads the potentiometer value.
4. **handleRoot()** builds the HTML page and inserts the sensor reading.
5. The meta refresh tag reloads the page every 2 seconds.
6. **server.on()** connects the root URL to handleRoot().
7. **server.begin()** starts the server; **server.handleClient()** processes requests.

Execution steps

Connect the potentiometer to GPIO 34, 3.3V and GND; enter Wi-Fi credentials; upload the program; open Serial Monitor at 115200 baud; copy the ESP32 IP address; open it in a browser on the same network; rotate the potentiometer and observe the reading.

Expected output

The browser displays a value such as **Sensor Value: 2356**. The page refreshes every two seconds. The value changes when the potentiometer is rotated.

Important clarification: This simplified program hosts a local webpage on the ESP32. It does not upload readings to an external cloud database. Your lab manual's described implementation is an ESP32 web server displaying the potentiometer value.

Practical 5 – Industrial Data Analysis Using Excel

Aim: Perform IoT data analysis using Excel charts and graphs.

Software and dataset

Item	Details
Software	Microsoft Excel
Dataset	Farm Temperature and Humidity
Source	Kaggle
Coverage	4 farms, approximately 1 km apart
Duration	3 months
Reading interval	Every 1 hour

Procedure

1. Download the Farm Temperature and Humidity dataset from Kaggle.
2. Open the file in Microsoft Excel.
3. Select the temperature column and the first 24-hour readings.
4. Choose **Insert** → **Recommended Charts** and select a suitable graph.
5. Repeat with 48-hour readings.
6. Repeat the same steps for humidity readings.
7. Use line charts or scatter plots to understand patterns.

Example Excel formulas

Formula	Purpose
=AVERAGE(A1:A24)	Calculates average
=MAX(A1:A24)	Finds maximum
=MIN(A1:A24)	Finds minimum
=SUM(A1:A24)	Calculates total

Observations from the lab manual

Graph / period	Observation
24-hour temperature	Peak around 12 o'clock; temperature falls later and at night.
48-hour temperature	Similar peaks around hour 12 and hour 36.
Humidity	The manual notes that humidity increases at night.
Humidity at hours 13 and 37	A similar fall is observed around these times.

How to explain this practical

"We downloaded farm temperature and humidity readings from Kaggle and analyzed them in Microsoft Excel. We created line charts and scatter plots to identify changes and patterns over time. The graphs showed temperature peaks around noon and generally higher humidity at night, according to the observations in our lab manual."

Viva points: Data analytics means collecting, processing and analyzing data to find useful patterns. A line chart shows trends over time; a scatter plot displays individual data points and possible relationships.

Quick Viva Revision

Common IIoT questions

Question	Short answer
What is IoT?	Internet of Things: connected physical devices that collect and exchange data.
What is IIoT?	Industrial Internet of Things: connected industrial machines, sensors and systems.
What is a sensor?	A device that detects a physical parameter.
What is an actuator?	A device that performs a physical action from a control signal.
What is ADC?	Analog-to-Digital Converter.
What is GPIO?	General Purpose Input/Output.
What is MQTT?	A lightweight messaging protocol used in IoT.
What is edge computing?	Processing data near where it is generated.
What is predictive maintenance?	Using data to anticipate equipment failure before it happens.
What is a Digital Twin?	A virtual representation of a physical object or system.

Important values to memorize

Practical	Remember
1	GPIO 34; ADC 0–4095; warning threshold > 2000; Serial 115200
2	PIR OUT → D2; LED → D13 in this simplified code; Serial 9600
3	LEDs → GPIO 12 and 14; 220Ω resistors; web server port 80; Serial 115200
4	Potentiometer → GPIO 34; WiFi.h; WebServer.h; port 80
5	Excel; Kaggle farm dataset; 4 farms; 3 months; hourly readings

Common troubleshooting

• If the ESP32 port is missing, check the USB data cable, driver and selected COM port.
• If Wi-Fi does not connect, check the SSID and password and ensure the network is available.
• If the webpage does not open, check that the phone/computer is on the same network and use the current IP printed in Serial Monitor.
• If the LED does not light, check polarity, GPIO pin, resistor and GND connection.
• If the potentiometer reading does not change, check its middle-pin connection to GPIO 34 and its 3.3V/GND supply.

Before the external exam

For each practical, be ready to explain: (1) the aim, (2) components used, (3) pin connections, (4) the main function in the code, and (5) the expected output. Understand the program rather than memorizing it line by line.

Source and scope

Prepared from the uploaded *Computer Laboratory II: Industrial Internet of Things* lab manual (Course 417526), especially experiment pages 11–25. The code is simplified for learning and demonstration; wiring should be checked against the actual board pin labels and component polarity before powering the circuit.